SYSTEM DESIGN FOR FRESHERS

The 12 concepts that actually show up in interviews

No fluff, no 50-page architecture diagrams. Just the concepts freshers get asked about, explained with companies you already use every day — plus where to go deeper for free.

BUILT FOR THE 2025–2026 GRADUATING BATCH

@techbyshreyas

BEFORE YOU START

A quick note

You don't need to become a system design expert before your first job. Nobody is asking a fresher to design Netflix from scratch in a 30-minute round. What actually happens is the interviewer picks ONE of these concepts and asks you to reason through it out loud.

This PDF has the twelve that come up again and again — each with a real example from an app you've actually used, because that's what makes it stick instead of sliding out of your head the next morning.

Read it once. Then try explaining each concept out loud — to yourself, to a friend, doesn't matter — without looking at the page. If you can do that, you're interview-ready on that topic. That's genuinely the whole test.

Keep doing DSA the way you already are. This just fills the other half of the interview that most freshers leave completely empty.

— Shreyas

01

Latency vs Throughput

Two different problems that freshers answer with the same line.

THE SIMPLE VERSION

Latency is how long ONE request takes to get a response. Throughput is how MANY requests your system can handle in a given time. A system can be fast for one user and still fall over when a thousand users hit it together — that's low latency, low throughput, happening at once.

WHERE YOU'VE ALREADY SEEN IT

Zepto's 10-minute delivery promise is a latency problem — get one order processed and out the door fast. The 8pm flash sale when half of Bangalore opens the app together is a throughput problem — the system has to survive volume, not just be quick for one person.

WHY THIS GETS ASKED

"How would you make this API faster" and "how would you handle 10 lakh concurrent users" sound similar but want completely different answers. Caching and better algorithms fix latency. Load balancers, horizontal scaling and queues fix throughput. Mixing the two up in an answer is one of the fastest ways to lose points.

GO DEEPER — FREE RESOURCES

→ ByteByteGo — "Latency vs Throughput" (YouTube, 4 min, free)

→ Gaurav Sen — Scalability playlist (YouTube)

→ donnemartin/system-design-primer on GitHub — search the page for "latency numbers every programmer should know"

02 CAP Theorem

Pick two — and in practice, the third one isn't really optional.

THE SIMPLE VERSION

In a distributed system (data spread across multiple machines), when a network partition happens — and it will — you have to choose between Consistency (everyone sees the same data) and Availability (the system keeps responding, even with possibly stale data). Partition tolerance isn't really a choice; networks fail, full stop. So really you're choosing C or A.

WHERE YOU'VE ALREADY SEEN IT

When a PhonePe or Paytm server in your region goes down, do they show you possibly outdated balance and let the transaction go through (choosing Availability), or do they block it with an error until they're sure (choosing Consistency)? Most payment systems lean CP because a wrong balance with real money is unacceptable. Instagram's like count leaning AP is fine — showing 4,532 instead of 4,533 for a second hurts nobody.

WHY THIS GETS ASKED

The classic version is "design X, would you go CP or AP and why." There's no universally correct answer — interviewers are checking if you can justify a trade-off with a real consequence, not whether you memorised the theorem.

GO DEEPER — FREE RESOURCES

→ Martin Kleppmann — free conference talks on YouTube, search "CAP theorem Kleppmann"

→ "Designing Data-Intensive Applications" by Martin Kleppmann, chapter 9 — the book most senior engineers point to, worth tracking down a copy

→ Gaurav Sen — CAP Theorem (YouTube)

03 Horizontal vs Vertical Scaling

Bigger machine, or more machines — and why the second one always wins eventually.

THE SIMPLE VERSION

Vertical scaling means upgrading the machine you already have — more RAM, more CPU, on the same server. Horizontal scaling means adding more machines and splitting the load across them. Vertical is simpler but has a hard ceiling — there's only so big one machine can get, and it's a single point of failure.

WHERE YOU'VE ALREADY SEEN IT

A college project running on one free-tier AWS instance is vertical-scaling territory — just bump the instance type when it slows down. Swiggy on the night of an IPL final can't buy one giant server fast enough; they horizontally scale, spinning up hundreds of smaller servers behind a load balancer instead.

WHY THIS GETS ASKED

"Traffic is about to grow 10x next month, what do you do" is almost always pointing at horizontal scaling. The follow-up they're listening for is whether you understand your servers now need to be stateless — no server should hold session data that only it knows about, or scaling out breaks things.

GO DEEPER — FREE RESOURCES

→ AWS Architecture Center — free whitepapers on scalability (aws.amazon.com/architecture)

→ freeCodeCamp — System Design Crash Course (YouTube)

→ ByteByteGo — Scaling video series

04 Load Balancing

The traffic cop nobody thinks about until it's missing.

THE SIMPLE VERSION

A load balancer sits in front of your servers and decides which one handles each incoming request, so no single server gets crushed while others sit idle. Common strategies: round robin (take turns), least connections (send to whichever server is least busy), and IP hash (same user keeps hitting the same server).

WHERE YOU'VE ALREADY SEEN IT

Opening Hotstar mid-match doesn't connect you to one fixed server. A load balancer — often something like Nginx or AWS's ELB — picks one of thousands of backend servers for your request, and a different one for the next person's.

WHY THIS GETS ASKED

The question rarely stops at "what is a load balancer." The real follow-up is "what if the load balancer itself crashes?" Good answer: run multiple load balancers behind DNS failover, so there's no single point of failure at the front door either. Freshers who only prepared the definition get caught here.

GO DEEPER — FREE RESOURCES

→ Nginx official docs — genuinely readable, free (nginx.org/en/docs)

→ Hussein Nasser — YouTube channel, load balancing playlist

→ Gaurav Sen — Load Balancer (YouTube)

05 Caching

Stop hitting the database for the same answer every single time.

THE SIMPLE VERSION

Caching means keeping frequently requested data somewhere much faster than your main database — usually in RAM, using tools like Redis or Memcached — so repeat requests don't have to go all the way to the slow disk-based DB. The hard part isn't adding a cache, it's keeping it in sync when the real data changes.

WHERE YOU'VE ALREADY SEEN IT

Flipkart's iPhone listing page during a sale gets hit by lakhs of people within seconds. Querying the database for every single page view would bring it down. Instead, the product details sit in a Redis cache and refresh every few seconds — the DB barely notices the traffic.

WHY THIS GETS ASKED

Interviewers love asking what happens when the underlying data changes but the cache still has the old value. That's the actual test — whether you know cache invalidation strategies (TTL expiry, write-through, cache-aside) and not just that "caching makes things fast."

GO DEEPER — FREE RESOURCES

→ Redis official docs + Redis University — free courses (university.redis.com)

→ ByteByteGo — "Caching Strategies" post/video

→ Gaurav Sen — Caching (YouTube)

06 Database Indexing

The index page of a book, applied to a database table.

THE SIMPLE VERSION

Without an index, a database scans every single row to find what you asked for. An index is a separate sorted structure (usually a B-Tree) pointing straight to the row you want, so lookups go from scanning millions of rows to nearly instant. The trade-off: every write now has to update the index too, and indexes take extra storage.

WHERE YOU'VE ALREADY SEEN IT

A hiring platform searching 2 crore candidate profiles by email — without an index on the email column, that's a full table scan, painfully slow. With an index, it's near-instant.

WHY THIS GETS ASKED

The gotcha question is "so should we just index every column?" The correct answer is no — more indexes mean slower inserts/updates and more storage used. Saying this out loud signals you understand trade-offs instead of treating indexing as a free win.

GO DEEPER — FREE RESOURCES

→ Use The Index, Luke — use-the-index-luke.com (the best free site on this topic, full stop)

→ PostgreSQL official docs — indexing section

→ freeCodeCamp — Database Indexing explained (YouTube)

07 Sharding / Partitioning

One giant database, cut into smaller pieces that can scale on their own.

THE SIMPLE VERSION

Sharding splits one large database into smaller pieces called shards, each holding a slice of the data, often on separate servers. Each shard has fewer rows to deal with, so queries are faster, and shards can scale independently.

WHERE YOU'VE ALREADY SEEN IT

Picture Zomato splitting its orders database by city — Mumbai orders on shard 1, Bangalore on shard 2. Queries get faster because each shard is smaller, and a busy city's shard can be scaled without touching the others.

WHY THIS GETS ASKED

"What if one city becomes way busier than the rest?" — that's the hot shard problem, and it's the standard follow-up. A solid fresher answer mentions re-sharding or picking a better shard key (hashing user_id instead of using city directly) so load spreads more evenly.

GO DEEPER — FREE RESOURCES

→ MongoDB docs — Sharding section (well explained even if you've never touched Mongo)

→ Hussein Nasser — Database Sharding (YouTube)

→ system-design-primer GitHub repo — sharding section

08 Concurrency vs Parallelism

One chef juggling three dishes vs three chefs, one dish each.

THE SIMPLE VERSION

Concurrency means dealing with multiple tasks by switching between them — not necessarily at the exact same instant. Parallelism means actually running multiple tasks at the same instant, usually on multiple CPU cores. Most backend servers use concurrency (async code) far more often than true parallelism.

WHERE YOU'VE ALREADY SEEN IT

A single Node.js server handling a thousand API requests using async/await is concurrency — it's switching between tasks, not running them all literally at once. Splitting that same workload across 8 CPU cores running simultaneously is parallelism.

WHY THIS GETS ASKED

People mix these up constantly and interviewers notice instantly. If you can also explain a race condition, a deadlock, and why locks/mutexes exist — in two sentences each — you're already ahead of most fresher candidates.

GO DEEPER — FREE RESOURCES

→ Hussein Nasser — Operating Systems / Concurrency series (YouTube)

→ freeCodeCamp — Operating System crash course (YouTube)

→ Gaurav Sen — Concurrency (YouTube)

09

Microservices vs Monolith

Starting with a monolith isn't a beginner mistake. It's usually correct.

THE SIMPLE VERSION

A monolith keeps the whole application — frontend logic, backend, business rules — in one codebase with one deployment. Microservices break the app into small independent services (auth, payments, notifications) that talk to each other over APIs, each deployable on its own.

WHERE YOU'VE ALREADY SEEN IT

Most college projects, and most startups on day one, are monoliths — and that's the right call, not a shortcut. Companies like Swiggy or Zomato moved toward microservices only once scale and team size actually demanded it.

WHY THIS GETS ASKED

Freshers often assume "microservices = automatically better." Interviewers like asking the reverse: "when would you NOT use microservices?" The strong answer is a small team, early-stage product, or unclear domain boundaries — start with a monolith, split later once it actually hurts.

GO DEEPER — FREE RESOURCES

→ Martin Fowler's blog — the original "Microservices" article (martinfowler.com, free)

→ ByteByteGo — Microservices vs Monolith (YouTube)

→ system-design-primer GitHub repo

10 Message Queues (Async Communication)

Don't make the user wait for things that don't need to happen right now.

THE SIMPLE VERSION

Instead of one service waiting for another to finish before responding, it drops a message in a queue and moves on immediately. The next service picks the message up whenever it's free. Tools like Kafka, RabbitMQ, or AWS SQS commonly sit in this role.

WHERE YOU'VE ALREADY SEEN IT

Placing an order on Amazon India doesn't run payment confirmation, SMS, email, and inventory update as one blocking chain. They're pushed onto queues so the checkout screen responds in well under a second, while the rest happens in the background.

WHY THIS GETS ASKED

Almost any "design a notification system" or "design an order processing flow" question expects a queue somewhere in the answer. Leaving it out is one of the more obvious gaps an interviewer will flag.

GO DEEPER — FREE RESOURCES

→ Apache Kafka — official "Introduction" docs (kafka.apache.org, surprisingly readable)

→ Gaurav Sen — Message Queues (YouTube)

→ Hussein Nasser — Kafka / queues playlist

11 Consistency Models

Does everyone see the truth instantly, or eventually?

THE SIMPLE VERSION

Strong consistency means the moment data is written, every read everywhere sees that latest value immediately. Eventual consistency allows a small lag — different servers might briefly disagree, but they converge to the same value shortly after.

WHERE YOU'VE ALREADY SEEN IT

A bank balance after a UPI transfer needs strong consistency — showing the old balance for even two seconds is a real problem. Instagram's follower count being eventually consistent — off by one for a moment — bothers nobody.

WHY THIS GETS ASKED

This connects straight back to CAP theorem. Explaining WHY a particular system picks eventual consistency (usually for speed and availability) is a stronger signal than just defining the two terms — it shows you understand the trade-off, not just the vocabulary.

GO DEEPER — FREE RESOURCES

→ Martin Kleppmann — free talks on consistency models (YouTube)

→ AWS — "Eventual Consistency" whitepaper, free on aws.amazon.com

→ ByteByteGo — Consistency Models explained

12 Design Patterns for LLD Rounds

System design at the code level — and where Razorpay-style LLD rounds live.

THE SIMPLE VERSION

System design (architecture) and design patterns (object-oriented code structure) are different but both come up. Four that show up constantly: Singleton (only one instance ever exists, e.g. a single DB connection pool), Factory (a method creates the right object without exposing how, e.g. returning a Razorpay, Paytm, or Stripe object based on input), Observer (objects subscribe to an event and get notified, e.g. how a follow notification reaches your phone), and Strategy (swap the algorithm at runtime, e.g. switching pricing logic for a premium user vs a regular one).

WHERE YOU'VE ALREADY SEEN IT

"Design a parking lot" or "design a vending machine" — common Low Level Design (LLD) interview questions — are really just applied design patterns wearing a costume.

WHY THIS GETS ASKED

At fresher level, knowing the four patterns above by name, with one real example each, covers most LLD rounds without needing the full Gang of Four catalogue.

GO DEEPER — FREE RESOURCES

→ refactoring.guru — the best free, visual explanation of design patterns

→ freeCodeCamp — Design Patterns crash course (YouTube)

→ Gaurav Sen — LLD playlist (YouTube)

LAST THING

Quick revision checklist

Go down this list the night before your interview. If you can explain any of these in under 30 seconds with an example, tick it off.

[ ] Latency vs Throughput

[ ] CAP Theorem

[ ] Horizontal vs Vertical Scaling

[ ] Load Balancing

[ ] Caching

[ ] Database Indexing

[ ] Sharding / Partitioning

[ ] Concurrency vs Parallelism

[ ] Microservices vs Monolith

[ ] Message Queues (Async Communication)

[ ] Consistency Models

[ ] Design Patterns for LLD Rounds

Saved this for later? That's exactly what it's for — bookmark it and come back the week of your interview.

More drops like this — DSA sheets, recruiter lists, roadmap PDFs — go out on @techbyshreyas first. Comment on the reel to get the next one straight in your DMs.